

CONFIDENTIAL DOCUMENT

Tawveeri PDR — Phase 1 & Phase 2 Status Report

Report Date: 2026-02-18

Document Type	Development Status Report
Project	Tawveeri — Price Comparison Platform
Prepared By	Etlag — etlag.sa
Version	1.0
Date	18 February 2026

About Etlag

Etlag is a Saudi-based software development company specializing in building modern digital platforms, web applications, and mobile solutions for businesses across the Kingdom. We transform ideas into scalable, production-grade products.

etlag.sa

Tawveeri PDR — Phase 1 & Phase 2 Status Report

Report Date: 2026-02-18 Branch: `phase2_v1` Compared Against: 10-Week Implementation Plan (PDR.pdf)

Overview

Per the PDR:

"Phase 1 focuses on growth and adoption, while Phase 2 introduces profitability through commissions from sales."

PHASE	SCOPE	WEEKS	IMPLEMENTED	PARTIAL	NOT DONE	COMPLETION
Phase 1	Growth & Adoption	1–5	29	1	0	98%
Phase 2	Profitability & Scale	6–10	14	13	12	53%

PHASE 1 — Growth & Adoption (Weeks 1–5)

Overall: 98%

The core platform is fully built. Users can register, search with category and spec filtering, compare prices, view products, save wishlists, and browse stores. All 5 store scrapers (cron + search) are operational. The only remaining partial item is custom daily backups (relies on Supabase managed backups).

Week 1 — Foundation & Setup 95%

#	TASK	STATUS	EVIDENCE
1	Development environment setup	✓	Next.js 15 + Turbopack, Tailwind v4, TypeScript
2	Database schema design (Users, Products, Stores, Transactions)	✓	<code>scripts/database/01-schema.sql</code> — 10+ tables
3	Server & API setup	✓	Next.js App Router API routes in <code>src/app/api/</code>
4	Security baseline (SSL, Encryption)	✓	Supabase SSL + <code>@supabase/ssr</code> secure auth
5	Daily backup system + Monitoring & Logging	●	Audit logging done (<code>src/lib/auth/audit.ts</code>). Daily backups rely on Supabase managed service — no custom backup system.
6	RLS (Row-Level Security) setup for database	✓	<code>scripts/database/02-rls-policies.sql</code>
7	User roles planning (admin, customer, store)	✓	4 roles in <code>src/lib/database/types.ts</code> ; middleware enforcement

Week 2 — User Accounts & Access 100%

#	TASK	STATUS	EVIDENCE
1	User registration (Email/ Phone/Google)	✓	Email+password, Phone OTP, Google/Facebook/Apple OAuth in <code>src/lib/auth/auth-context.tsx</code>
2	Guest access	✓	Search, browse, compare all work without auth
3	Account management (profile editing, password reset, delete account)	✓	<code>src/app/[locale]/(dashboard)/profile/page.tsx</code> — avatar, name, password change, account deletion
4	Legal compliance (Saudi data privacy laws)	✓	Bilingual Privacy Policy + Terms of Service in <code>(public)/privacy/</code> and <code>(public)/terms/</code>
5	User/Admin/Customer page creation (frontend access layer)	✓	Route groups <code>(public)</code> , <code>(dashboard)</code> , <code>/admin/</code> , <code>/store/</code> with middleware role checks

#	TASK	STATUS	EVIDENCE
6	Phone-based password reset	✓	3-step flow in <code>forgot-password/page.tsx</code> : phone input → OTP verification → new password. Uses <code>POST /api/auth/reset-password-phone</code> with OTP validation + <code>supabase.auth.admin.updateUserById()</code>

Week 3 — Search & Filtering 95%

#	TASK	STATUS	EVIDENCE
1	Product search by category (TV, Laptop, Smartphone)	✓	Category passed end-to-end: SearchPage → <code>POST /api/search/scrape</code> → <code>searchAllStores()</code> → filtered by <code>matchesCategory()</code> in <code>src/lib/scraping/Utils/category-utils.ts</code>
2	Filtering by brand, model	✓	Dynamic brand checkboxes from DB in <code>src/components/search/filter-sidebar.tsx</code>
3	Filtering by RAM, storage, size, resolution, color	✓	Dynamic spec filters per category in <code>filter-sidebar.tsx</code> using configs from <code>src/lib/scraping/config/spec-configs.ts</code> . Specs extracted client-side from product titles via <code>extractSpecsFromTitle()</code> . Covers RAM, storage, screen size, resolution, panel type, audio type, wireless/ANC for 6 categories.
4	Dynamic filters (based on category)	✓	Spec filter sections change dynamically based on selected category (smartphone shows RAM/storage, laptop shows CPU/GPU, TV shows resolution/panel, etc.). Discount, condition, and shipping speed filters also wired.
5	Sort by price, popularity, rating, store	✓	4 sort options + 7 filter types with URL state in <code>search/page.tsx</code>
6	Search suggestions (auto-suggest)	✓	Debounced autocomplete from DB products in <code>search-bar.tsx</code>

Week 4 — Store Integration 95%

#	TASK	STATUS	EVIDENCE
1	Integration with 5 stores (Extra, Jarir, Almanea, Noon, Amazon.sa)	✓	All 5 stores fully operational. Search scrapers work for all 4 main stores. Cron scrapers implemented for all 5: <code>JarirScraper</code> , <code>NoonScraper</code> (JSON API), <code>AmazonScraper</code> (cheerio + Puppeteer), <code>ExtraScraper</code> (<code>__NEXT_DATA__</code> + HTML fallback), <code>AlmaneaScraper</code> (HTML + JSON-LD). Orchestrator routes all stores in <code>scraping-orchestrator.ts</code> .
2	Localized store integration (additional Saudi stores)	✓	5 Saudi stores with bilingual configs (<code>name_ar</code> , <code>name_en</code>) in <code>src/lib/scraping/config/store-configs/</code>
3	Store onboarding portal (self-service)	✓	Full portal: <code>/store/dashboard</code> , <code>/store/products</code> , <code>/store/products/new</code> , <code>/store/analytics</code> , <code>/store/transactions</code>
4	Store rating system	✓	<code>store_reviews</code> table with multi-field ratings (delivery, quality, service); review form + display on store detail pages

Week 5 — Product Experience 95%

#	TASK	STATUS	EVIDENCE
1	Product detail page (images, specs, prices)	✓	Image gallery with carousel + full-screen modal, specs, multi-store pricing (<code>products/[slug]/page.tsx</code> — 864 lines)
2	Product availability checker	✓	4 statuses: <code>in_stock</code> , <code>out_of_stock</code> , <code>limited_stock</code> , <code>pre_order</code> with colored badges
3	Product videos/demos	✓	<code>ProductVideoPlayer</code> component — YouTube embeds + direct video files
4	Price comparison engine	✓	<code>ComparisonCard</code> with per-store pricing, best-price badges, savings calculation, affiliate links
5	Price history tracking	✓	<code>price_history</code> table + custom SVG chart with 30/90/365-day ranges and trend indicators
6	Multi-language support (Arabic/English)	✓	19 translation files, <code>SimpleIntlProvider</code> , RTL/LTR auto-switching

#	TASK	STATUS	EVIDENCE
7	Currency support (SAR)	✓	<code><Price></code> component with SAR SVG symbol; <code>formatPrice()</code> utilities in <code>src/lib/utls.ts</code>

Phase 1 — Functional Requirements Summary

REQUIREMENT	STATUS
User Registration & Auth (email, phone, social)	✓
Guest Access	✓
Product Search by category	✓
Brand & Model Filtering	✓
Advanced Filtering (specs)	✓
Price Comparison Engine	✓
Sort & Filter Results	✓
Store Integration (5 stores)	✓
Redirection to Store	✓
Product Detail Page	✓
Wishlist / Save Products	✓
Notification System (in-app)	✓
Multi-language (AR/EN)	✓
Currency Handling (SAR)	✓
Search Suggestions	✓
User Reviews & Ratings	✓
Product Availability Checker	✓
Price History Tracking	✓
Store Onboarding Portal	✓
Store Rating System	✓
Product Videos/Demos	✓
Mobile Responsiveness	✓
Search by Voice	✓
Barcode/QR Code Scanner	✓
Search History Log	✓
Multiple Login Options	✓
Legal Compliance Pages	✓
Totals	25 ✓ · 0 ● · 0 ✗

Phase 1 — Remaining Work

All Phase 1 gaps have been resolved:

GAP	STATUS	RESOLUTION
~~Complete cron scrapers for Amazon, Noon, Extra, Almanea~~	✓ Done	All 5 store scrapers implemented (<code>noon-scraper.ts</code> , <code>amazon-scraper.ts</code> , <code>extra-scraper.ts</code> , <code>almanea-scraper.ts</code>) + orchestrator updated
~~Add spec-based filter UI (RAM, storage, screen size, color)~~	✓ Done	Dynamic spec filters per category in <code>filter-sidebar.tsx</code> with <code>extractSpecsFromTitle()</code> client-side extraction. 6 categories covered.
~~Make category selection actually filter results~~	✓ Done	Category passed through SearchPage → API → <code>searchAllStores()</code> → <code>matchesCategory()</code> filter
~~Phone-based password reset~~	✓ Done	3-step OTP flow: phone → verify → new password. API route <code>reset-password-phone/route.ts</code> + updated <code>forgot-password/page.tsx</code>

PHASE 2 — Profitability & Scale (Weeks 6–10)

Overall: 53%

Monetization infrastructure (affiliate tracking, commission calculation) is solid. The biggest gaps are in engagement features (email/push/coupons), AI recommendations, SEO, mobile app, and production-readiness.

Week 6 — Personalization & Favorites 55%

#	TASK	STATUS	EVIDENCE
1	Wishlist / Save products	✓	Full CRUD with notes in <code>(dashboard)/wishlist/page.tsx</code>
2	Favorites sync (cross-device)	✓	Supabase-backed; auto-syncs on login from any device
3	Search history log	✓	Auto-logged to <code>search_history</code> table; shown in search bar + saved searches feature
4	Personalized recommendations	●	Category-based + view_count algorithm on dashboard. Not sophisticated.
5	AI-powered smart recommendations	✗	No ML models, collaborative filtering, or AI infrastructure

Week 7 — Notifications & Engagement 50%

#	TASK	STATUS	EVIDENCE
1	Notification system (price drop, back in stock)	✓	Full in-app system with bilingual content; cron job checks price alerts (<code>/api/cron/check-price-alerts/</code>)
2	Email/SMS alerts	●	HTML email templates built for 7 types. No email provider connected — calls <code>supabase.functions.invoke('send-email')</code> placeholder. SMS = OTP only (Authentic).
3	Push notifications (mobile)	●	Preference toggles exist in settings UI. No service worker, no Web Push API, no FCM backend.
4	Daily deals section	✓	Full deals page with search, sorting, stats cards, expiry tracking (<code>(public)/deals/page.tsx</code>)
5	Coupon integration	✗	No coupon table, no promo code UI, no validation logic

Week 8 — Monetization & Analytics 60%

#	TASK	STATUS	EVIDENCE
1	Premium store listings	●	<code>is_premium</code> + <code>is_featured</code> DB fields exist; store listing filters by premium/featured. No upgrade/payment flow.
2	Loyalty program / Cashback	✗	Zero implementation — no tables, no UI, no logic
3	Store dashboard (merchant analytics)	✓	Revenue charts, product performance, click/conversion rates, transaction history in <code>/store/dashboard/</code> and <code>/store/analytics/</code>
4	Admin analytics dashboard	✓	KPIs, revenue charts, user growth, category distribution, search analytics, activity logs in <code>/admin/dashboard/</code> and <code>/admin/analytics/</code>
5	SEO optimization	●	Root metadata in <code>layout.tsx</code> . Missing: dynamic per-page metadata, <code>sitemap.xml</code> , <code>robots.txt</code> , JSON-LD structured data, OG tags

Week 9 — Advanced Comparison 70%

#	TASK	STATUS	EVIDENCE
1	Comparison tables (specs & prices)	✓	Up to 4 products side-by-side; dynamic spec rows with 40+ translated keys in <code>(dashboard)/compare/page.tsx</code>

#	TASK	STATUS	EVIDENCE
2	Comparison across stores (warranty, shipping, return policies)	●	DB fields exist (<code>warranty_info_ar/en</code> , <code>return_policy_ar/en</code> , <code>delivery_info_ar/en</code>). <code>ComparisonCard</code> shows warranty/delivery icons. Full policy comparison UI not built.
3	Delivery time comparison	●	<code>delivery_time_days</code> and <code>delivery_cost</code> fields in DB; tracked in cart items. Limited end-user UI exposure.
4	Multi-store cart	✓	Fully implemented: store-based grouping, quantity management, gift wrapping in <code>src/lib/cart/</code>
5	Gift option integration	✓	Gift wrapping toggle + message + Web Share API in <code>src/components/products/gift-option.tsx</code>

Week 10 — Performance, Compliance & Launch 25%

#	TASK	STATUS	EVIDENCE
1	Mobile app launch (iOS & Android)	✗	No React Native / Expo / mobile app codebase
2	Mobile optimization (responsive, feature parity)	✓	Extensive Tailwind responsive breakpoints, mobile dialogs, responsive grids
3	Performance optimization (< 3s search)	●	Turbopack in dev. No Lighthouse audit, bundle analysis, or load time benchmarks.
4	Scalability (1M+ users)	●	Supabase is horizontally scalable. No load testing or CDN setup evidence.
5	Reliability (99.9% uptime)	✗	No monitoring service (Sentry, Datadog, UptimeRobot, etc.)
6	Cross-browser compatibility	●	Feature detection for modern APIs; no polyfills for legacy browsers
7	Accessibility compliance (WCAG 2.1)	●	Radix UI baseline; <code>prefers-reduced-motion</code> . Missing ARIA landmarks, comprehensive labels, contrast audit.
8	Energy efficiency optimization	✗	No specific infrastructure optimization
9	Legal compliance (Saudi regulations)	✓	Privacy + Terms (bilingual)
10	Localization (Hijri/Gregorian + Arabic numerals)	✗	Gregorian only; no Hijri calendar or Eastern Arabic numeral support
11	Full Go-Live	✗	Not launched

Phase 2 — Functional Requirements Summary

REQUIREMENT	STATUS
Comparison Table (side-by-side specs & prices)	✓
Commission Tracking (click → conversion)	✓
Affiliate Tracking (unique links per redirect)	✓
Admin Analytics Dashboard	✓
Store Dashboard (merchant analytics)	✓
Store Onboarding Portal	✓
Wishlist / Favorites	✓
Favorites Sync (cross-device)	✓
Search History Log	✓
Daily Deals Section	✓
Multi-Store Cart	✓
Gift Option Integration	✓
Mobile Responsiveness	✓
Legal Compliance (Saudi laws)	✓
Notification System (in-app)	✓

REQUIREMENT	STATUS
Personalized Recommendations	●
Email/SMS Alerts	●
Push Notifications	●
Premium Store Listings	●
SEO Optimization	●
Comparison (warranty, returns, delivery)	●
Delivery Time Comparison	●
Performance (< 3s search)	●
Scalability (1M+ users)	●
Cross-Browser Compatibility	●
Accessibility (WCAG 2.1)	●
Monitoring & Logging (external)	●
AI Smart Recommendations	✗
Coupon Integration	✗
Loyalty Program / Cashback	✗
Mobile App (iOS & Android)	✗
Reliability (99.9% uptime monitoring)	✗
Data Backup (custom daily)	✗
Energy Efficiency	✗
Localization (Hijri + Arabic numerals)	✗
Single Sign-On (corporate/academic)	✗
Full Go-Live	✗
Totals	15 ✓ · 12 ● · 10 ✗

Phase 2 — Remaining Work (Priority Order)

#	GAP	IMPACT	EFFORT	NOTES
1	Connect email service (Resend/SendGrid)	Critical	Low	Templates + helpers already built. Just need to wire up a provider.
2	SEO fundamentals — sitemap, robots.txt, dynamic metadata, JSON-LD	Critical	Medium	No organic search traffic without this.
3	Coupon integration — DB table, admin UI, user display	High	Medium	Listed as PDR requirement, zero implementation.
4	Push notifications — service worker + Web Push API	High	Medium	Settings UI exists, backend missing.
5	Premium store upgrade flow — payment integration	High	High	DB fields exist; needs payment gateway.
6	AI recommendations	Medium	High	Could start with simple collaborative filtering before full ML.
7	Monitoring — Sentry / Datadog / UptimeRobot	Medium	Low	Required for production reliability.
8	Accessibility audit — WCAG 2.1 AA	Medium	Medium	Radix helps; needs ARIA labels, landmarks, contrast fixes.
9	Loyalty / Cashback program	Medium	High	Zero foundation — needs design + full implementation.
10	Hijri calendar + Arabic numerals	Low	Medium	Saudi-specific localization.
11	Mobile app (React Native / Expo)	Low (short-term)	Very High	Entire new codebase. Consider after web is production-ready.

Visual Summary

Phase 1 – Growth & Adoption

98%

Week 1	Foundation	95%
Week 2	Accounts	100%
Week 3	Search	95%
Week 4	Stores	95%
Week 5	Products	95%

Phase 2 – Profitability & Scale

53%

Week 6	Personal.	55%
Week 7	Notifications	50%
Week 8	Monetization	60%
Week 9	Comparison	70%
Week 10	Launch	25%

Legend:  = Implemented |  = Partial |  = Not Implemented